

# Maps and Hash Tables

Data Structures

Chapter 10.1 - 10.2 in textbook

Based in part on textbook slides



# Maps

- A **map** is an ADT for a searchable collection of **key-value** item pairs
  - A map is also often also called a *dictionary* or an *associative array*.
  - Multiple items with the *same key* are **not** permitted.
- Very common that data of all sorts is organized as *key-value* pairs:
  - Example are records on persons, where the *social-security-number* (*isl. kennitala*) is the *key*, the the remaining personal information the *value*.
- The fundamental operations on a *map* ADT are:
  - **insert** (key, value)
  - **delete** (key)
  - **search** (key)
  - **iterate** (over keys)
    - Sorted on key or not?
    - Insertion order preserved or not?

**Question:** What is the Icelandic name for a map?

# Maps in Python

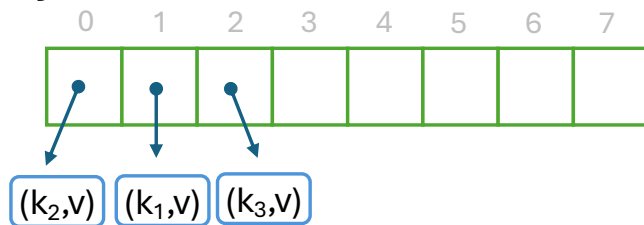
- The **dict** (dictionary) data type in Python implements a map ADT.

| Operation       | Description                                                                                                                                                                                                              |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>M[k]</b>     | Return the value $v$ associated with key $k$ in map $M$ , if one exists; otherwise raise a <i>KeyError</i> . In Python, this is implemented with the special method <code>__getitem__</code> .                           |
| <b>M[k] = v</b> | Associate value $v$ with key $k$ in map $M$ , replacing the existing value if the map already contains an item with key equal to $k$ . In Python, this is implemented with the special method <code>__setitem__</code> . |
| <b>del M[k]</b> | Remove from map $M$ the item with key equal to $k$ ; if $M$ has no such item, then raise a <i>KeyError</i> . In Python, this is implemented with the special method <code>__delitem__</code> .                           |
| <b>iter(M)</b>  | The default iteration for a map generates a sequence of keys in the map (preserving insertion order from version 3.7 onwards). In Python, this is implemented with the special method <code>__iter__</code> .            |

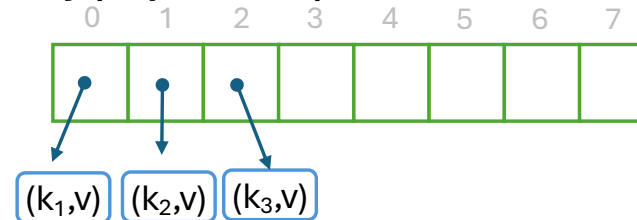
# Implementing a Map

- How to implement a map?
  - Which *data-structure* allows the most efficient implementation?
  - Assume insertion order:  $k_2, k_1, k_3$  and that  $k_1 < k_2 < k_3$ .

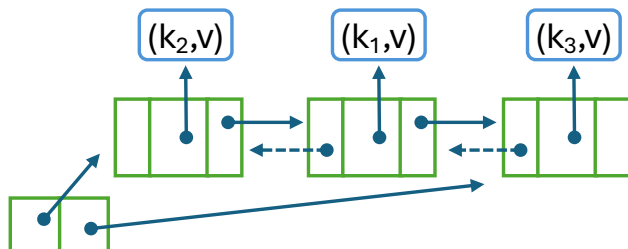
Array:



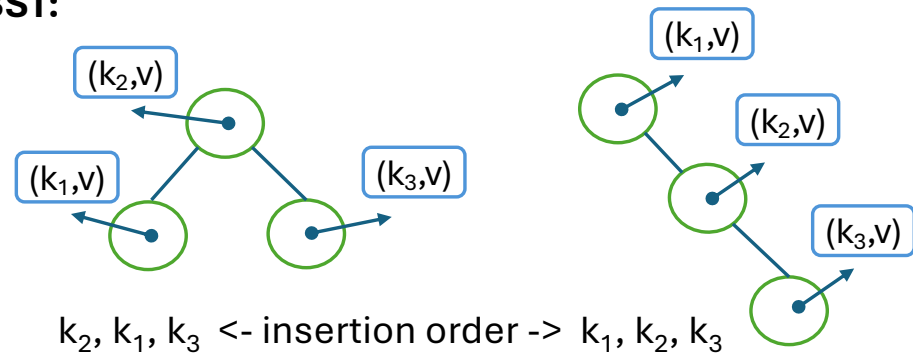
Array (kept sorted):



Linked list:



BST:



# Average-Case Time Complexity

| Operation | Array    | Array (sorted) | Linked List | BST                | In My Wildest Dreams!      |
|-----------|----------|----------------|-------------|--------------------|----------------------------|
| search    | $O(n)$   | $O(\log_2 n)$  | $O(n)$      | $O(\log_2 n)^{**}$ | <b><math>O(1)</math></b>   |
| insert    | $O(1)^*$ | $O(n)$         | $O(1)$      | $O(\log_2 n)^{**}$ | <b><math>O(1)^*</math></b> |
| update    | $O(n)$   | $O(\log_2 n)$  | $O(n)$      | $O(\log_2 n)^{**}$ | <b><math>O(1)</math></b>   |
| delete    | $O(n)$   | $O(\log_2 n)$  | $O(n)$      | $O(\log_2 n)^{**}$ | <b><math>O(1)</math></b>   |
| iter      | $O(n)$   | $O(n)$         | $O(n)$      | $O(n)$             | <b><math>O(n)</math></b>   |

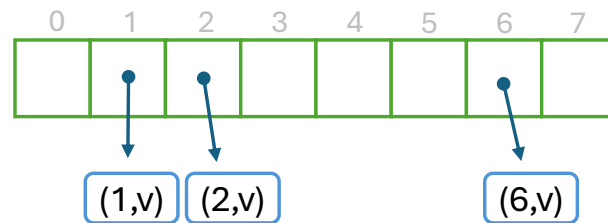
\* Amortized

\*\* Assuming reasonably balanced

# Making Dreams Come True!

- **Small dreams**

- The *key* is always a small positive integer, say in range  $[0, N-1]$



- **Big dreams**

- The *key* can be of **any** data-type (even strings) and take **any** value.
- What if would have a **function** what could convert any *object* into an *integer* in the range  $[0, N-1]$ ?

# Hash Function

- A **hash function**  $h$  maps a key of a given type to an integers in an interval  $[0, N - 1]$
- Example:

$$h(x) = x \bmod N$$

is a hash function for *integer* keys

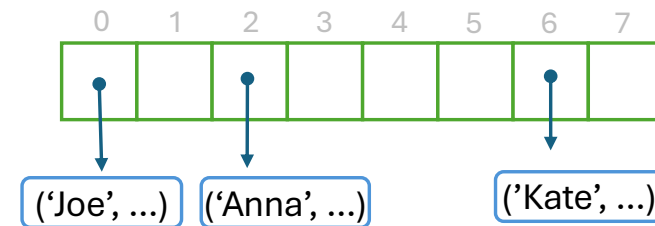
- The integer  $h(x)$  is called the **hash value** of key  $x$
- A hash function  $h(x)$  is usually specified as the composition of two functions:
  - **Hash-code:**  $h_1: \text{key} \rightarrow \text{integer}$ 
    - The goal is to “disperse” the keys in a “random” way.
  - **Compression:**  $h_2: \text{integer} \rightarrow [0, N - 1]$ 
    - The goal is to “compress” the hash code integer into the range  $[0, N - 1]$
- The hash-code is applied first, and then the compression function:  $h(x) = h_2(h_1(x))$

# Hash Table

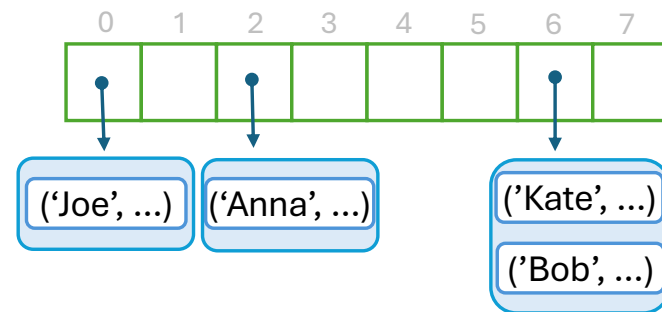
- A **hash table** for a given key type consists of
  - A hash function  $h$
  - An array (called table) of size  $N$
- When implementing a map with a hash table, the goal is to store item:
  - $(k, v)$  at index  $i = h(k)$
- A **collision** occurs when two different keys have the same hash value.
  - A collision resolution scheme called **separate chaining** uses **buckets** (containers) for resolving this.

- Assume:

- $h(\text{'Joe'}) = 0$ ,  $h(\text{'Kate'}) = 6$ , and  $h(\text{'Anna'}) = 2$

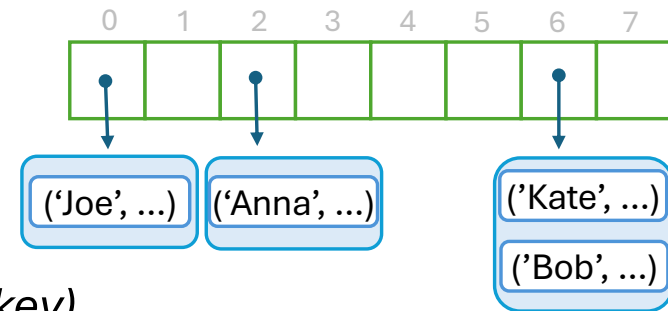


- Now add  $h(\text{'Bob'}) = 6$



# Hash Table (cont.)

- Assume  $T$  is a *hash table* using:
  - *separate chaining*,
  - an *array* of size  $N$ ,
  - Assuming a good (disperse) hash function  $h(\text{key})$
- What is the *expected* size of a bucket, when inserting  $n$  items into  $T$ ?
- The expected bucket size,  $\lambda = n/N$ , is called the **load-factor**
  - It should be bounded by a (preferably small  $< 1$ ) constant, such that operations within a bucket are all  $O(1)$ .
  - *As long as  $\lambda$  is  $O(1)$ , the core operations on the hash table run in  $O(1)$  expected time.*
- We ensure this by **resizing** the hash-table (e.g., by a factor of two) when the load-factor exceeds **some preset constant**.
  - We **must rehash all the elements** into the new array when resizing.



Question: Why do you think that is?

# Average-Case Time Complexity

| Operation | Array    | Array (sorted) | Linked List | BST                | Hash Table                 |
|-----------|----------|----------------|-------------|--------------------|----------------------------|
| search    | $O(n)$   | $O(\log_2 n)$  | $O(n)$      | $O(\log_2 n)^{**}$ | <b><math>O(1)</math></b>   |
| insert    | $O(1)^*$ | $O(n)$         | $O(1)$      | $O(\log_2 n)^{**}$ | <b><math>O(1)^*</math></b> |
| update    | $O(n)$   | $O(\log_2 n)$  | $O(n)$      | $O(\log_2 n)^{**}$ | <b><math>O(1)</math></b>   |
| delete    | $O(n)$   | $O(\log_2 n)$  | $O(n)$      | $O(\log_2 n)^{**}$ | <b><math>O(1)</math></b>   |
| iter      | $O(n)$   | $O(n)$         | $O(n)$      | $O(n)$             | <b><math>O(n)</math></b>   |

\* Amortized

\*\* Assuming reasonably balanced

# Implement a (custom) Map in Python

- You can implement your own custom map in Python :
  - Subclass *MutableMapping*
  - Implement:
    - `__getitem__`    # `v = M[k]`
    - `__setitem__`    # `M[k] = v`
    - `__delitem__`    # `del M[k]`
    - `__iter__`        # `for k in M ...`
    - `__len__`         # `len(M)`
  - You get the remaining dict operations “for free”
    - Base class implements in terms of those above.
- Generally speaking, unnecessary to do so as **dict()** is quite sophisticated.

```
from collections.abc import MutableMapping

class MyDict(MutableMapping):
    def __getitem__(self, key):
        ...
    def __setitem__(self, key, value):
        ...
    def __delitem__(self, key):
        ...
    def __iter__(self):
        ...
    def __len__(self):
        ...
```

# Summary

- A **map** is a fundamental and widely used abstract data type (ADT).
  - For unsorted maps, most often implemented using a *hash-table*
    - Core operations, average time-complexity  **$O(1)$**
  - For sorted maps, most often implemented using some sort of balanced trees
    - Core operations, average time-complexity  $O(\log n)$
- Most programming languages support *maps* in various capacity:
  - Python: **dict** (unsorted, insertion order)
  - JavaScript: **Map** (unsorted, insertion order)
  - C++: **std::map** (sorted)



Image created using ChatGPT